An Analysis of Flash attention

Flash attention

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Flash attention -- Part 1

Masked attention The transformer architecture is constructed to calculate output tokens iteratively. Assuming t = 0 {\displaystyle t=0} refers to the calculation of the first output token i = 0 {\displaystyle i=0} , for step t > 0 {\displaystyle t>0} , the output token i = 0 {\displaystyle i=0} shall remain constant. This ensures properties of the model similar to autoregressive models. Therefore, at every time step t {\displaystyle t} , the calculation for all outputs i {\displaystyle i} should not have access to tokens at position j {\displaystyle j} for j >= i {\displaystyle j>=i} (as it naturally is the case for time step t = i {\displaystyle t=i} , when tokens j > t {\displaystyle j>t} are not yet calculated). This behavior may be accomplished before the softmax stage by adding a mask matrix M {\displaystyle M} that is − ∞ {\displaystyle -\infty } at entries where the attention link must be cut, and 0 {\displaystyle 0} at other places: MaskedAttention (Q , K , V) = softmax (M + Q K T d k) V {\displaystyle {\begin{aligned}{\text{MaskedAttention}}(Q,K,V)={\text{softmax}}\left(M+{\frac {QK^{\mathrm {T} }}{\sqrt {d_{k}}}}\right)V\end{aligned}}} The following matrix is commonly used in decoder self-attention modules, called "causal masking": M causal = [0 − ∞ − ∞ … − ∞ 0 0 − ∞ … − ∞ 0 0 0 … − ∞ ■ ■ ■ ■ ■ 0 0 0 … 0] {\displaystyle M_{\text{causal}}={\begin{bmatrix}0&-\infty &-\infty &\dots &-\infty \\0&0&-\infty &\dots &-\infty \\0&0&0&\dots &-\infty \\\vdots &\vdots &\vdots &\ddots &\vdots \\0&0&0&\dots &0\end{bmatrix}}}

Flash attention -- Part 2

In general, there are 3 classes of language modelling tasks: "masked", "autoregressive", and "prefixLM". These classes are independent of a specific modeling architecture such as transformer, but they are often discussed in the context of transformer. In a masked task, one or more of the tokens is masked out, and the model would produce a probability distribution predicting what the masked-out tokens are based on the context. The loss function for the task is typically sum of log-perplexities for the masked-out tokens: Loss = − ∑ t ∈ masked tokens ln ■ (probability of t conditional on its context) {\displaystyle {\text{Loss}}=-\sum _{t\in {\text{masked tokens}}}\ln({\text{probability of }}t{\text{ conditional on its context}})} and the model is trained to minimize this loss function. The BERT series of models are trained for masked token prediction and another task. In an autoregressive task, the entire sequence is masked at first, and the model produces a probability distribution for the first token. Then the first token is revealed and the model predicts the second token, and so on. The loss function for the task is still typically the same. The GPT series of models are trained by autoregressive tasks. In a prefixLM task, the sequence is divided into two parts. The first part is presented as context, and the model predicts the first token of the second part. Then that would be revealed, and the model predicts the second token, and so on. The loss function for the task is still typically the same. The T5 series of models are trained by prefixLM tasks. Note that "masked" as in "masked language modelling" is not "masked" as in "masked attention", and "prefixLM" as in "prefix language modeling" is not "prefixLM" as in " prefix language model".

Flash attention -- Part 3

Multimodality Transformers can also be used/adapted for modalities (input or output) beyond just text, usually by finding a way to "tokenize" the modality. Multimodal models can either be trained from scratch, or by finetuning. A 2022 study found that transformers pretrained only on natural language can be finetuned on only 0.03% of parameters and become competitive with LSTMs on a variety of logical and visual tasks, demonstrating transfer learning. The LLaVA was a vision-language model composed of a language model (Vicuna-13B) and a vision model (ViT-L/14), connected by a linear layer. Only the linear layer is finetuned. Vision transformers adapt the transformer to computer vision by breaking down input images as a series of patches, turning them into vectors, and treating them like embedding vector of tokens in a standard transformer. Conformer and later Whisper follow the same pattern for speech recognition, first turning the speech signal into a spectrogram, which is then treated like an image, i.e. broken down into a series of patches, turned into vectors and treated like embedding vector of tokens in a standard transformer. Perceivers are a variant of transformers designed for multimodality. For image generation, notable architectures are DALL-E 1 (2021), Parti (2022), Phenaki (2023), and Muse (2023). Unlike later models, DALL-E is not a diffusion model. Instead, it uses a decoder-only transformer that autoregressively generates a text, followed by the token representation of an image, which is then converted by a variational autoencoder to an image. Parti is an encoder–decoder transformer, where the encoder processes a text prompt, and the decoder generates a token representation of an image. Muse is an encoder-only transformer that is trained to predict masked image tokens from unmasked image tokens. During generation, all input tokens are masked, and the highest-confidence predictions are included for the next iteration, until all tokens are predicted. Phenaki is a

text-to-video model. It is a bidirectional masked transformer conditioned on pre-computed text tokens. The generated tokens are then decoded to a video.

Flash attention -- Part 4

machine translation time series prediction document summarization document generation named entity recognition (NER) writing computer code based on requirements expressed in natural language. speech-to-text Beyond traditional NLP, the transformer architecture has had success in other applications, such as:

Flash attention -- Part 5

MultiheadAttention (Q , K , V) = Concat i ∈ [n heads] (Attention (X W i Q , X W i K , X W i V)) W O {\displaystyle {\text{MultiheadAttention}}(Q,K,V)={\text{Concat}}_{i\in [n_{\text{heads}}]}\left({\text{Attention}}(XW_{i}^{Q},XW_{i}^{K},XW_{i}^{V})\right)W^{O}} with Multi-Query Attention, there is just one W K , W V {\displaystyle W^{K},W^{V}} , thus:

Flash attention -- Part 6

/* third sublayer */ z_d_copy ← copy(z_d) for each t in 1:length(z_d) do z_d[t] ← layer.layer_norm(z_d[t]) z_d ← layer.feedforward(z_d) for each t in 1:length(z_d) do z_d[t] ← z_d[t] + z_d_copy[t]

Flash attention -- Part 7

Speculative decoding is a method to accelerate token decoding. Similarly to speculative execution in CPUs, future tokens are computed quickly, then verified. If the quickly computed tokens are incorrect, they are discarded and computed slowly. The key factor in speculative decoding is that a transformer decoder can verify faster than it can decode, in the following sense. Suppose we have two transformer models like GPT-3 and GPT-3-small, both with a context window size of 512. To generate an entire context window autoregressively with greedy decoding with GPT-3, it must be run for 512 times, each time generating a token x 1 , x 2 , . . . , x 512 {\displaystyle x_{1},x_{2},...,x_{512}} , taking time 512 T GPT-3 {\displaystyle 512T_{\text{GPT-3}}} . However, if we had some educated guess for the values of these tokens, we could verify all of them in parallel, in one run of the model, by checking that each x t {\displaystyle x_{t}} is indeed the token with the largest log-likelihood in the t {\displaystyle t} -th output. In speculative decoding, a smaller model or some other simple heuristic is used to generate a few speculative tokens that are subsequently verified by the larger model. For example, suppose we use GPT-3-small to generate four speculative tokens: x ~ 1 , x ~ 2 , x ~ 3 , x ~ 4 {\displaystyle {\tilde {x}}_{1},{\tilde {x}}_{2},{\tilde {x}}_{3},{\tilde {x}}_{4}} . This only takes 4 T GPT-3-small {\displaystyle 4T_{\text{GPT-3-small}}} . These tokens are then run through the larger GPT-3 in one go. Suppose that x ~ 1 {\displaystyle {\tilde {x}}_{1}} and x ~ 2 {\displaystyle {\tilde {x}}_{2}} are verified by GPT-3 as what it would have picked, then those are kept, but x ~ 3 {\displaystyle {\tilde {x}}_{3}} is not, so x ~ 3 , x ~ 4 {\displaystyle {\tilde {x}}_{3},{\tilde {x}}_{4}} are discarded, and GPT-3 is run on those. This would take 4 T GPT-3-small + 3 T GPT-3 {\displaystyle 4T_{\text{GPT-3-small}}+3T_{\text{GPT-3}}} , which might be shorter than 4 T GPT-3 {\displaystyle 4T_{\text{GPT-3}}} . For non-greedy decoding, similar ideas apply, except the speculative tokens are accepted or rejected stochastically, in a way that guarantees the final output distribution is the same as if speculative decoding was not used.